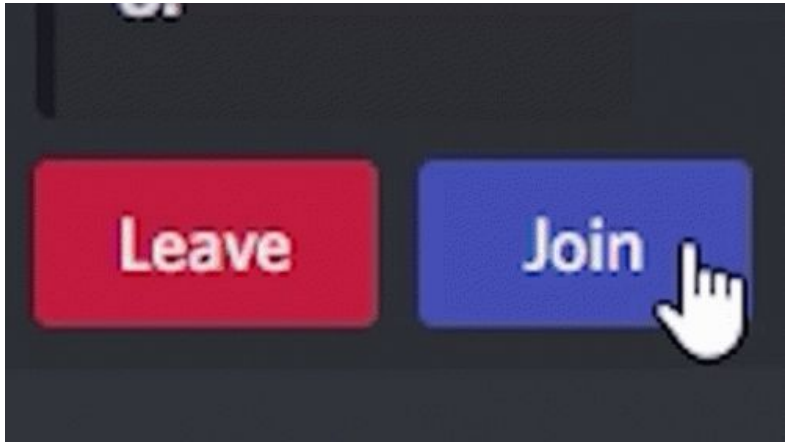
Newton School of Technology

7

Neural Networks

Join the Lecture on NS Portal :)

Newton School of Technology



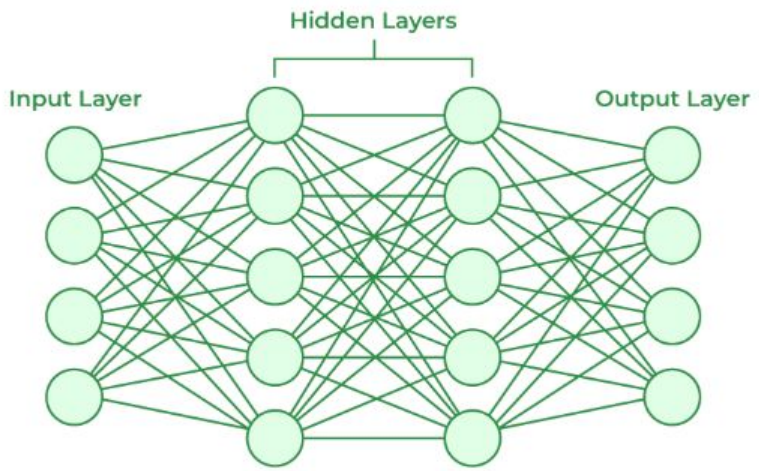
Positioning Neural Networks

Newton School of Technology

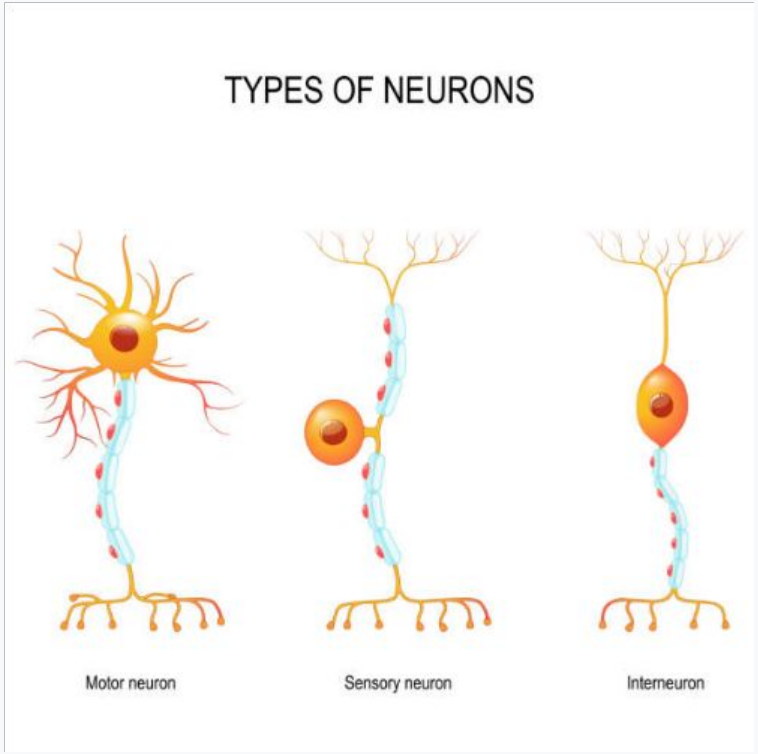
- **AI (Artificial Intelligence):** The broadest term. Machines mimicking human intelligence.
- **ML (Machine Learning):** A subset of AI. Algorithms that improve through experience.
- **DL (Deep Learning):** A subset of ML. Neural networks with many layers.

Why "Deep"?

Refers to the hierarchy of representations (many layers). Examples: Self-driving cars, ChatGPT, Medical Imaging.



Biological Inspiration



Anatomy of a Neuron

- **Dendrites (Input):** Receives signals from other neurons.
- **Soma (Processing):** The cell body that processes information.
- **Axon (Output):** Transmits electrical impulses.
- **Synapses (Weights):** Connection points where signal strength varies.

Key Concept: Artificial neurons are mathematical simplifications mimicking this signal transmission process.

The Artificial Neuron

Newton School of Technology

Components

- **Inputs (x):** The data features.
- **Weights (w):** Importance of each input.
- **Bias (b):** Activation threshold.

The Process

$$y = f \sum_{i=1}^{n} w_i x_i + b$$

Activation Function (f): Decides if the neuron "fires".

The Single Perceptron

Newton School of Technology

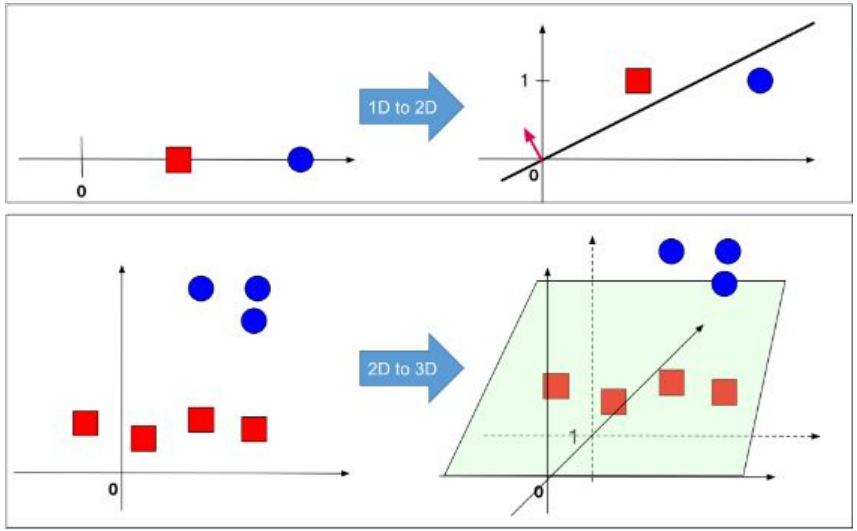
Rosenblatt's Perceptron (1958)

The first algorithmic model of a neuron, designed for image recognition. It is fundamentally a **Linear Classifier**.

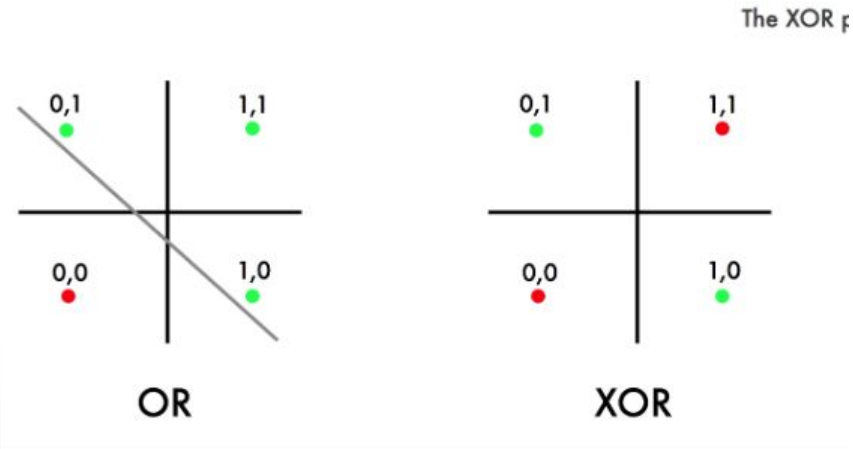
Binary Classification

It finds a straight line (or hyperplane) that separates data into two classes (0 or 1).

If $\sum wx + b > 0$, Output 1.
Otherwise, Output 0.



Limitations: The XOR Problem



Linear Separability

A single perceptron can model **AND** and **OR** gates because their outputs can be separated by a single straight line.

The Failure of XOR

XOR (Exclusive OR) outputs 1 only if inputs are different. As seen in the graph, no single straight line can separate the classes.

Impact: This observation by Minsky and Papert (1969) led to the first "AI Winter".

Multi-Layer Perceptrons (MLP)

Newton School of Technology

The Solution: Hidden Layers

By stacking neurons in layers, we create a Multi-Layer Perceptron.

The layers between Input and Output are called **Hidden Layers**.

This allows the network to learn non-linear boundaries.

Universal Approximation Theorem

A network with just one hidden layer and enough neurons can approximate any continuous function.

- Input Layer
- Hidden Layer(s) (Feature extraction)
- Output Layer (Prediction)

Forward Propagation

Newton School of Technology

Data moves from input to output. Each neuron calculates its weighted sum and applies an activation.

1. Input

Features (X) are fed into the network.

2. Weighted Sum

Linear transformation: $Z = WX + B$

3. Activation

Non-linear transformation: $A = f(Z)$

Activation Functions: The Classics

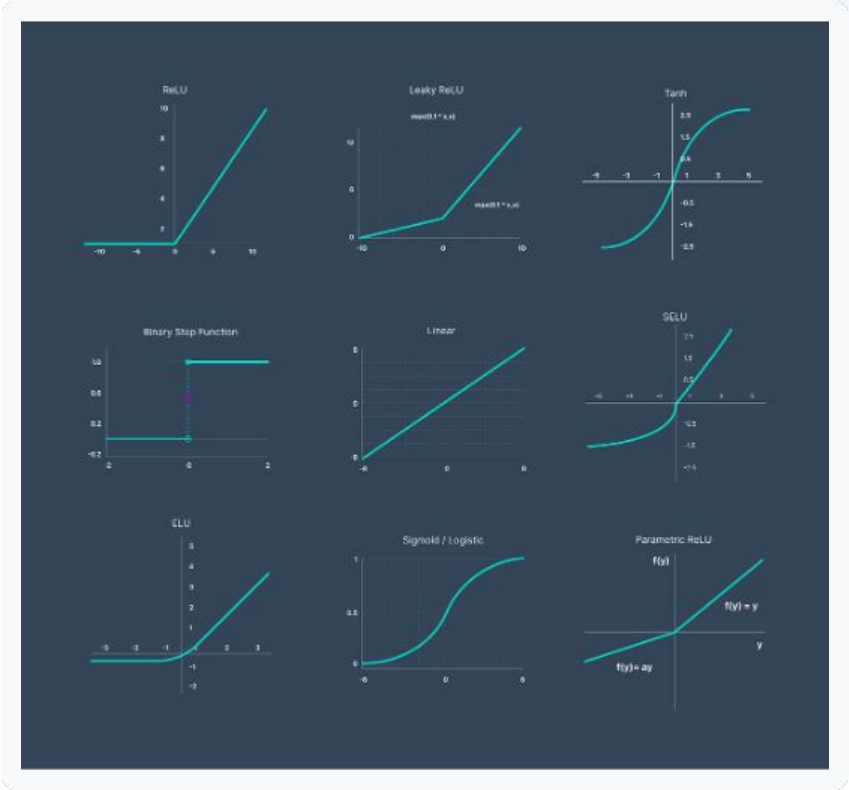
Newton School of Technology

Sigmoid

- Range: (0, 1)
- Use: Binary classification (probability).
- **Cons:** "Vanishing Gradient" problem.

Tanh (Hyperbolic Tangent)

- Range: (-1, 1)
- Pros: Zero-centered.
- **Cons:** Still suffers from vanishing gradients.



Modern Activation Functions

Newton School of Technology

ReLU (Rectified Linear Unit)

Status: The industry standard for hidden layers.

Pros: Fast, solves vanishing gradient for positive values.

Cons: "Dead ReLU" (neurons stuck at 0).

Softmax

Use: Output layer for Multi-Class Classification.

Effect: Converts raw scores (logits) into probabilities that sum to 1.

```
[2.0, 1.0, 0.1] -> [0.7, 0.2, 0.1]
```

Loss Functions (The Scorecard)

Newton School of Technology

A Loss Function measures "How wrong is the model?". The goal is to minimize this value.

MSE (Mean Squared Error)

Used for **Regression** (predicting numbers).

Calculates the squared difference between prediction and actual value.

Cross-Entropy Loss

Used for **Classification** (predicting categories).

Penalizes confident wrong predictions heavily.

The Optimizer: Gradient Descent

Newton School of Technology

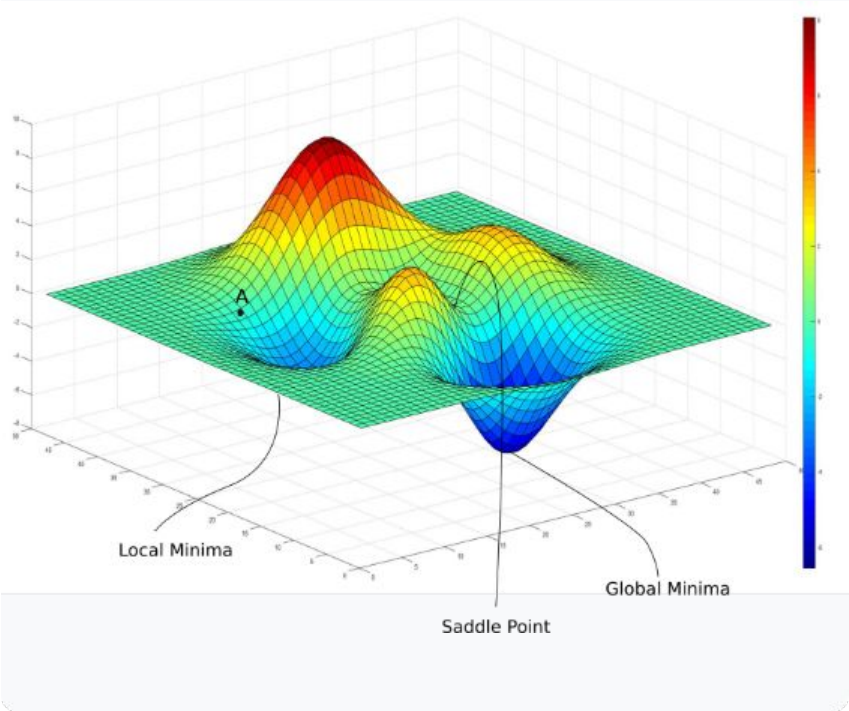
The Analogy

Imagine you are on a mountain (Loss landscape) in thick fog.

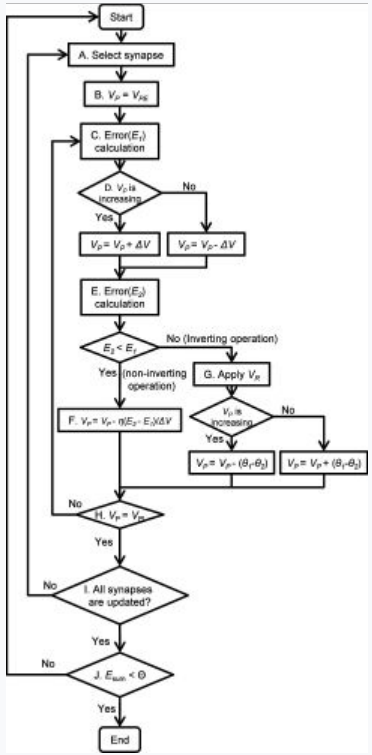
Strategy: Feel the slope under your feet (the Gradient) and take a small step downhill. Repeat until you reach the valley.

The Formula

New Weight = Old Weight - (Learning Rate × Gradient)



Backpropagation (The Engine)



Distributing the Blame

We know the final error. But which weights caused it?

Chain Rule of Calculus

Backpropagation uses the Chain Rule to compute gradients layer by layer, starting from the Output and moving backward to the Input.

Goal: Calculate dL/dw (How much loss changes with respect to weight).

Training Terminology

Newton School of Technology

Epoch

One complete pass through the entire training dataset.

Batch Size

Number of training samples processed before the model updates its weights.

Iteration

Number of batches needed to complete one epoch. (Total Data / Batch Size).

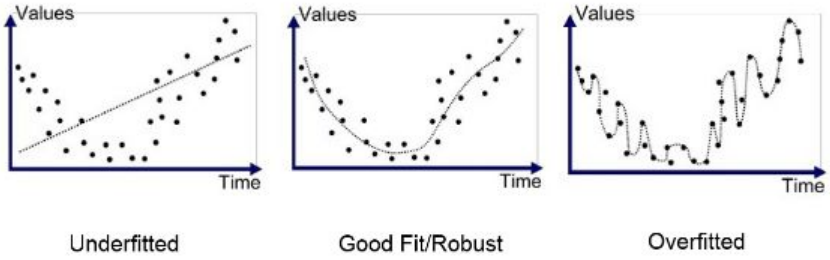
Overfitting

Memorization vs. Generalization

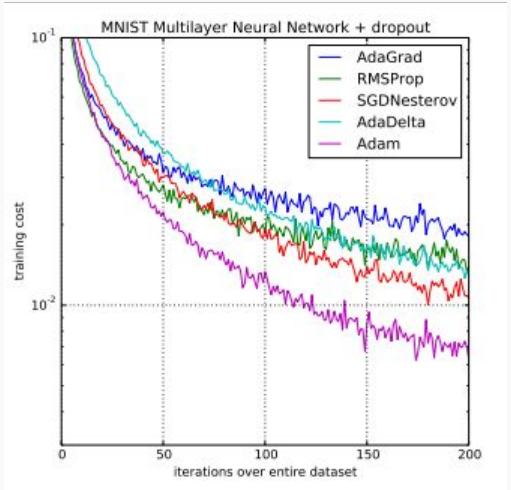
Overfitting: The model learns the noise in the training data instead of the actual pattern. It performs perfectly on training data but fails on new data.

Signs

- Training Loss: Decreases continually.
- Validation Loss: Starts decreasing, then increases (gap widens).



Optimizers Beyond SGD



SGD

The basic standard. Can be slow and get stuck in local minima.

Momentum

Accumulates velocity in the relevant direction. Dampens oscillations.

Adam (Adaptive Moment Estimation)

Combines Momentum with adaptive learning rates. The default choice for most problems.

Limitations of MLPs

Newton School of Technology

Standard dense layers struggle with spatial data (images) and sequential data (text) because they ignore structure.

CNNs (Convolutional Neural Networks)

Use filters to capture spatial patterns.

Perfect for: Computer Vision.

RNNs / Transformers

Designed for sequence data and memory.

Perfect for: NLP and Time Series.

Standard Workflow in Code

1. Data Preparation

Load dataset, normalize, split into Train/Validation, create DataLoaders.

2. Define Architecture

Create a class defining layers and forward pass.

3. Setup Training

Choose Loss Function (e.g., CrossEntropy) and Optimizer (e.g., Adam).

4. Training Loop

Iterate epochs, compute loss, backpropagate, update weights.

```
train_datagen = ImageDataGenerator(
rotation_range = 15,
    width_shift_range = 0.1,
    height_shift_range = 0.1,
    shear_range = 0.1,
    zoom_range = 0.1,
    horizontal_flip = True,
    fill_mode = "nearest")

# train the network
model.compile(optimizer = "Adam", loss = "categorical_crossentropy", metrics = [
    "accuracy"])


# using early stopping to exit training if validation loss is not decreasing
earlystopping = EarlyStopping(monitor = 'val_loss', mode = 'min', verbose = 1,
    patience = 20)

# save the best model with lower validation loss
checkpointer = ModelCheckpoint(filepath = "trafficSign_weights.hdf5", verbose =
    1, save_best_only=True)


history = model.fit(train_datagen.flow(X_train, y_train, batch_size=64),
    validation_data=(X_val, y_val), steps_per_epoch=len(X_train) // 64,
    epochs= 2, callbacks=[checkpointer, earlystopping])
```

Common Coding Interview Questions

Newton School of Technology

Q1: Zeroing Gradients

Why do we do optimizer.zero_grad() inside the loop?

```
optimizer.zero_grad() loss.backward() optimizer.step()
```

Answer: PyTorch accumulates gradients by default. We must clear them before each update step to avoid mixing gradients from different batches.

Q2: Define a Simple Layer

How do you define a fully connected layer in PyTorch?

```
import torch.nn as nn fc_layer = nn.Linear(in_features=10,
out_features=5)
```

Context: Maps 10 inputs to 5 outputs. It automatically initializes weights and biases.

Advanced Coding Patterns

Newton School of Technology

Q3: Moving to GPU

How do you ensure your code runs on GPU if available?

```
device = "cuda" if torch.cuda.is_available() else "cpu"
model.to(device) inputs = inputs.to(device)
```

Q4: Reshaping for FC Layers

How do you flatten a 28x28 image before a linear layer?

```
# x shape: [batch, 1, 28, 28] x = x.view(x.size(0), -1) # New shape:
[batch, 784]
```

```
class ODESolver(nn.Module):
    def __init__(self, model, dt):
        super(ODESolver, self).__init__()
        self.model = model
        self.dt = dt
    def forward(self, x):
        with torch.enable_grad():
            outputs = []
            for i, layer in
enumerate(self.model):
                outputs.append(layer(x))
                x = outputs[-1]
        return x
    def loss(self, x, t):
        with torch.enable_grad():
            outputs = []
            for i, layer in
enumerate(self.model):
                outputs.append(layer(x))
                x = outputs[-1]
        return x
```

Practical Concepts Check

Newton School of Technology

Bias Term

Q: Why is a bias term necessary?

A: It allows the activation function to shift, fitting data that doesn't pass through the origin.

Last Layer Activation

Q: For 10-class probability output?

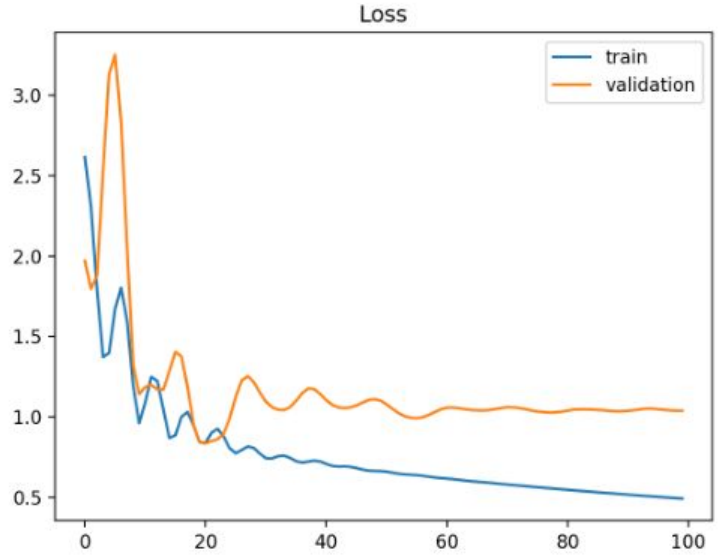
A: Softmax. It ensures all outputs sum to 1.0.

Learning Rate Zero

Q: What if Learning Rate is 0?

A: The model never learns. Weights never update because the step size is zero.

Visual Diagnostics: Loss Curves



Diagnosing Learning Rates

- **High LR (Yellow):** Loss fluctuates wildly or diverges. The steps are too big to settle in the valley.
- **Low LR (Blue):** Loss decreases very slowly. It will take too long to converge.
- **Good LR (Red):** Steady, consistent decrease in loss over time.

Vanishing Gradient Check

If the gradients in early layers are near zero (seen via tensor.grad), your network isn't learning features. **Solution:** Switch to ReLU.

Scenario: Debugging Overfitting

The Situation

You are training a network to classify cats vs. dogs.

- Training Accuracy: **99%**
- Testing Accuracy: **55%**

Diagnosis: Massive Overfitting.

Solutions

1. **Get More Data:** More examples help the model generalize better.
2. **Data Augmentation:** Flip/rotate existing images to create "new" data.
3. **Reduce Model Complexity:** Use fewer layers or fewer neurons per layer.
4. **Early Stopping:** Stop training when validation loss starts rising.

Newton School of Technology

Thank You

Questions?